11.3 AVL Trees

The TreeMap class, which uses a standard binary search tree as its data structure, should be an efficient map data structure, but its worst-case performance for the various operations is linear time, because it is possible that a series of operations results in a tree with linear height. In this section, we describe a simple balancing strategy that guarantees worst-case logarithmic running time for all the fundamental map operations.

Definition of an AVL Tree

The simple correction is to add a rule to the binary search tree definition that will maintain a logarithmic height for the tree. Although we originally defined the height of a subtree rooted at position p of a tree to be the number of *edges* on the longest path from p to a leaf (see Section 8.1.3), it is easier for explanation in this section to consider the height to be the number of *nodes* on such a longest path. By this definition, a leaf position has height 1, while we trivially define the height of a "null" child to be 0.

In this section, we consider the following ***height-balance property***, which characterizes the structure of a binary search tree T in terms of the heights of its nodes.

***Height-Balance Property*:** For every position p of T, the heights of the children of p differ by at most 1.

Any binary search tree T that satisfies the height-balance property is said to be an ***AVL tree***, named after the initials of its inventors: Adel'son-Vel'skii and Landis. An example of an AVL tree is shown in Figure 11.11.

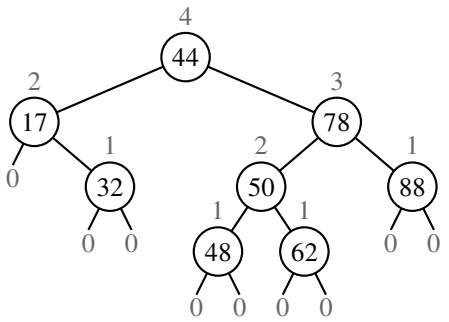


Figure 11.11: An example of an AVL tree. The keys of the items are shown inside the nodes, and the heights of the nodes are shown above the nodes (with empty subtrees having height 0).

An immediate consequence of the height-balance property is that a subtree of an AVL tree is itself an AVL tree. The height-balance property has also the important consequence of keeping the height small, as shown in the following proposition.

Proposition 11.2: *The height of an AVL tree storing n entries is $O(\log n)$.*

Justification: Instead of trying to find an upper bound on the height of an AVL tree directly, it turns out to be easier to work on the "inverse problem" of finding a lower bound on the minimum number of nodes $n(h)$ of an AVL tree with height h. We will show that $n(h)$ grows at least exponentially. From this, it will be an easy step to derive that the height of an AVL tree storing n entries is $O(\log n)$.

We begin by noting that $n(1) = 1$ and $n(2) = 2$, because an AVL tree of height 1 must have exactly one node and an AVL tree of height 2 must have at least two nodes. Now, an AVL tree with the minimum number of nodes having height h for $h \geq 3$, is such that both its subtrees are AVL trees with the minimum number of nodes: one with height $h-1$ and the other with height $h-2$. Taking the root into account, we obtain the following formula that relates $n(h)$ to $n(h-1)$ and $n(h-2)$, for $h \geq 3$:

$$n(h) = 1 + n(h-1) + n(h-2). \tag{11.1}$$

At this point, the reader familiar with the properties of Fibonacci progressions (Section 1.8 and Exercise C-3.49) will already see that $n(h)$ is a function exponential in h. To formalize that observation, we proceed as follows.

Formula 11.1 implies that $n(h)$ is a strictly increasing function of h. Thus, we know that $n(h-1) > n(h-2)$. Replacing $n(h-1)$ with $n(h-2)$ in Formula 11.1 and dropping the 1, we get, for $h \geq 3$,

$$n(h) \; > \; 2 \cdot n(h-2). \tag{11.2}$$

Formula 11.2 indicates that $n(h)$ at least doubles each time h increases by 2, which intuitively means that $n(h)$ grows exponentially. To show this fact in a formal way, we apply Formula 11.2 repeatedly, yielding the following series of inequalities:

$$\begin{aligned} n(h) &> 2 \cdot n(h-2) \\ &> 4 \cdot n(h-4) \\ &> 8 \cdot n(h-6) \\ &\;\;\vdots \\ &> 2^i \cdot n(h-2i). \end{aligned} \tag{11.3}$$

That is, $n(h) > 2^i \cdot n(h-2i)$, for any integer i, such that $h-2i \geq 1$. Since we already know the values of $n(1)$ and $n(2)$, we pick i so that $h-2i$ is equal to either 1 or 2. That is, we pick

$$i = \left\lceil \frac{h}{2} \right\rceil - 1.$$

By substituting the above value of i in Formula 11.3, we obtain, for $h \geq 3$,

$$\begin{aligned} n(h) &> 2^{\lceil \frac{h}{2} \rceil - 1} \cdot n\left(h - 2\left\lceil \frac{h}{2} \right\rceil + 2\right) \\ &\geq 2^{\lceil \frac{h}{2} \rceil - 1} n(1) \\ &\geq 2^{\frac{h}{2}-1}. \end{aligned} \tag{11.4}$$

By taking logarithms of both sides of Formula 11.4, we obtain

$$\log(n(h)) > \frac{h}{2} - 1,$$

from which we get

$$h < 2\log(n(h)) + 2, \tag{11.5}$$

which implies that an AVL tree storing n entries has height at most $2\log n + 2$. ■

By Proposition 11.2 and the analysis of binary search trees given in Section 11.1, the operation `__getitem__`, in a map implemented with an AVL tree, runs in time $O(\log n)$, where n is the number of items in the map. Of course, we still have to show how to maintain the height-balance property after an insertion or deletion.

11.3.1 Update Operations

Given a binary search tree T, we say that a position is ***balanced*** if the absolute value of the difference between the heights of its children is at most 1, and we say that it is ***unbalanced*** otherwise. Thus, the height-balance property characterizing AVL trees is equivalent to saying that every position is balanced.

The insertion and deletion operations for AVL trees begin similarly to the corresponding operations for (standard) binary search trees, but with post-processing for each operation to restore the balance of any portions of the tree that are adversely affected by the change.

Insertion

Suppose that tree T satisfies the height-balance property, and hence is an AVL tree, prior to the insertion of a new item. An insertion of a new item in a binary search tree, as described in Section 11.1.3, results in a new node at a leaf position p. This action may violate the height-balance property (see, for example, Figure 11.12a), yet the only positions that may become unbalanced are ancestors of p, because those are the only positions whose subtrees have changed. Therefore, let us describe how to restructure T to fix any unbalance that may have occurred.

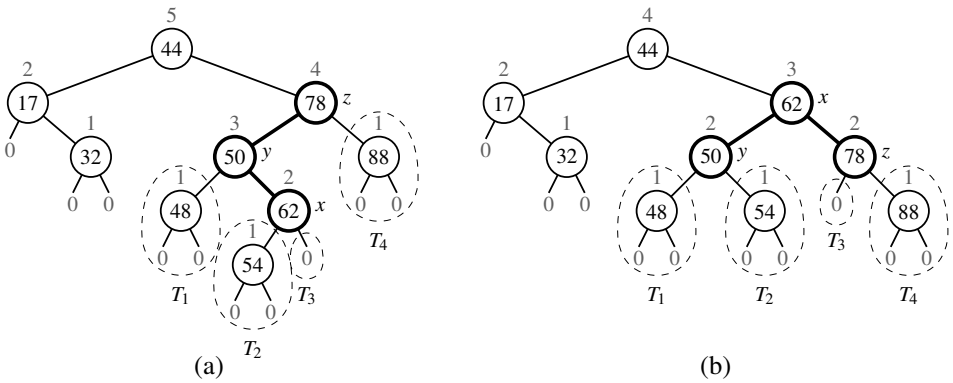


Figure 11.12: An example insertion of an item with key 54 in the AVL tree of Figure 11.11: (a) after adding a new node for key 54, the nodes storing keys 78 and 44 become unbalanced; (b) a trinode restructuring restores the height-balance property. We show the heights of nodes above them, and we identify the nodes x, y, and z and subtrees T_1, T_2, T_3, and T_4 participating in the trinode restructuring.

We restore the balance of the nodes in the binary search tree T by a simple "search-and-repair" strategy. In particular, let z be the first position we encounter in going up from p toward the root of T such that z is unbalanced (see Figure 11.12a.) Also, let y denote the child of z with higher height (and note that y must be an ancestor of p). Finally, let x be the child of y with higher height (there cannot be a tie and position x must also be an ancestor of p, possibly p itself). We rebalance the subtree rooted at z by calling the ***trinode restructuring*** method, `restructure`(x), originally described in Section 11.2. An example of such a restructuring in the context of an AVL insertion is portrayed in Figure 11.12.

To formally argue the correctness of this process in reestablishing the AVL height-balance property, we consider the implication of z being the nearest ancestor of p that became unbalanced after the insertion of p. It must be that the height of y increased by one due to the insertion and that it is now 2 greater than its sibling. Since y remains balanced, it must be that it formerly had subtrees with equal heights, and that the subtree containing x has increased its height by one. That subtree increased either because $x = p$, and thus its height changed from 0 to 1, or because x previously had equal-height subtrees and the height of the one containing p has increased by 1. Letting $h \geq 0$ denote the height of the tallest child of x, this scenario might be portrayed as in Figure 11.13.

After the trinode restructuring, we see that each of x, y, and z has become balanced. Furthermore, the node that becomes the root of the subtree after the restructuring has height $h+2$, which is precisely the height that z had before the insertion of the new item. Therefore, any ancestor of z that became temporarily unbalanced becomes balanced again, and this one restructuring restores the height-balance property ***globally***.

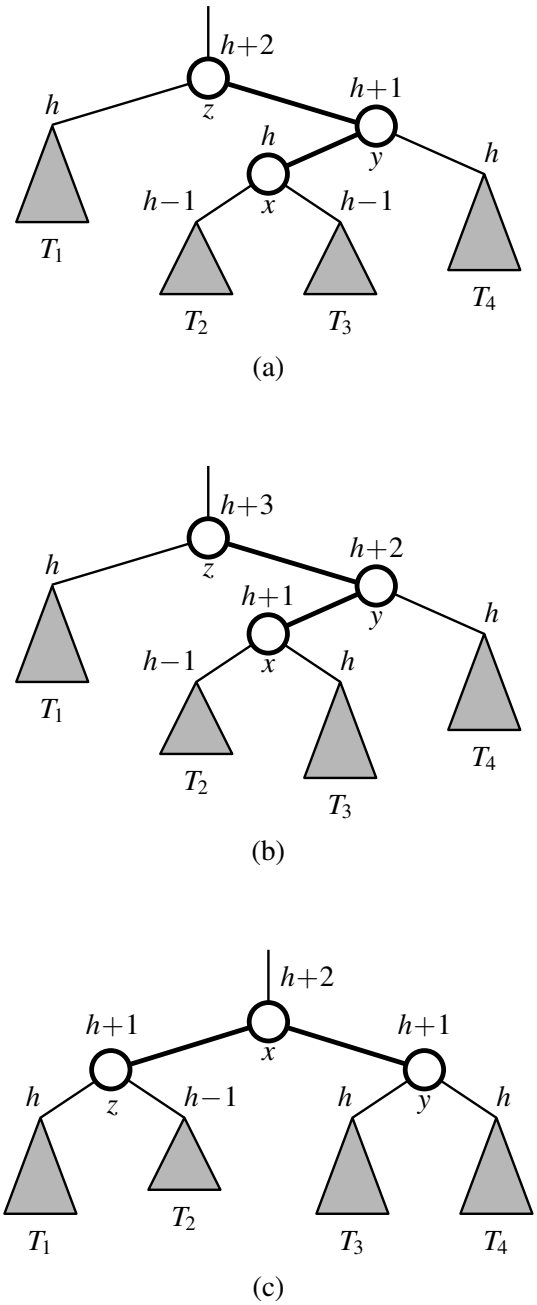


Figure 11.13: Rebalancing of a subtree during a typical insertion into an AVL tree: (a) before the insertion; (b) after an insertion in subtree T_3 causes imbalance at z; (c) after restoring balance with trinode restructuring. Notice that the overall height of the subtree after the insertion is the same as before the insertion.

Deletion

Recall that a deletion from a regular binary search tree results in the structural removal of a node having either zero or one children. Such a change may violate the height-balance property in an AVL tree. In particular, if position p represents the parent of the removed node in tree T, there may be an unbalanced node on the path from p to the root of T. (See Figure 11.14a.) In fact, there can be at most one such unbalanced node. (The justification of this fact is left as Exercise C-11.49.)

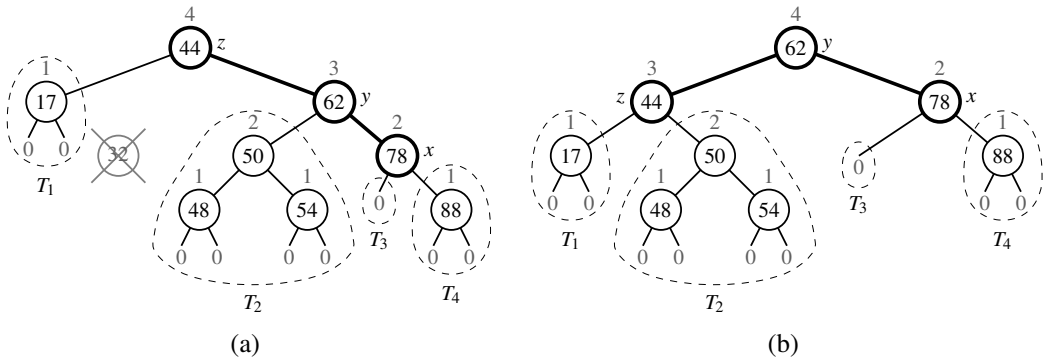


Figure 11.14: Deletion of the item with key 32 from the AVL tree of Figure 11.12b: (a) after removing the node storing key 32, the root becomes unbalanced; (b) a (single) rotation restores the height-balance property.

As with insertion, we use trinode restructuring to restore balance in the tree T. In particular, let z be the first unbalanced position encountered going up from p toward the root of T. Also, let y be the child of z with larger height (note that position y is the child of z that is not an ancestor of p), and let x be the child of y defined as follows: If one of the children of y is taller than the other, let x be the taller child of y; else (both children of y have the same height), let x be the child of y on the same side as y (that is, if y is the left child of z, let x be the left child of y, else let x be the right child of y). In any case, we then perform a $\mathsf{restructure}(x)$ operation. (See Figure 11.14b.)

The restructured subtree is rooted at the middle position denoted as b in the description of the trinode restructuring operation. The height-balance property is guaranteed to be ***locally*** restored within the subtree of b. (See Exercises R-11.11 and R-11.12). Unfortunately, this trinode restructuring may reduce the height of the subtree rooted at b by 1, which may cause an ancestor of b to become unbalanced. So, after rebalancing z, we continue walking up T looking for unbalanced positions. If we find another, we perform a restructure operation to restore its balance, and continue marching up T looking for more, all the way to the root. Still, since the height of T is $O(\log n)$, where n is the number of entries, by Proposition 11.2, $O(\log n)$ trinode restructurings are sufficient to restore the height-balance property.

Performance of AVL Trees

By Proposition 11.2, the height of an AVL tree with n items is guaranteed to be $O(\log n)$. Because the standard binary search tree operation had running times bounded by the height (see Table 11.1), and because the additional work in maintaining balance factors and restructuring an AVL tree can be bounded by the length of a path in the tree, the traditional map operations run in worst-case logarithmic time with an AVL tree. We summarize these results in Table 11.2, and illustrate this performance in Figure 11.15.

Operation	Running Time
k in T	$O(\log n)$
T[k] = v	$O(\log n)$
T.delete(p), del T[k]	$O(\log n)$
T.find_position(k)	$O(\log n)$
T.first( ), T.last( ), T.find_min( ), T.find_max( )	$O(\log n)$
T.before(p), T.after(p)	$O(\log n)$
T.find_lt(k), T.find_le(k), T.find_gt(k), T.find_ge(k)	$O(\log n)$
T.find_range(start, stop)	$O(s+\log n)$
iter(T), reversed(T)	$O(n)$

Table 11.2: Worst-case running times of operations for an n-item sorted map realized as an AVL tree T, with s denoting the number of items reported by find_range.

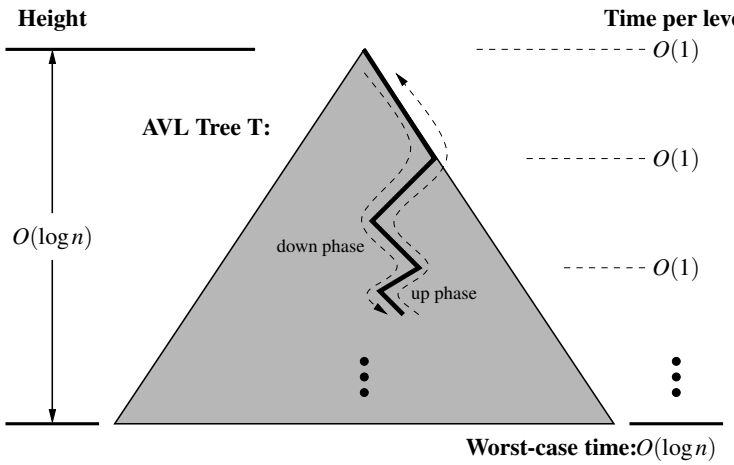


Figure 11.15: Illustrating the running time of searches and updates in an AVL tree. The time performance is $O(1)$ per level, broken into a down phase, which typically involves searching, and an up phase, which typically involves updating height values and performing local trinode restructurings (rotations).

11.3.2 Python Implementation

A complete implementation of an AVLTreeMap class is provided in Code Fragments 11.12 and 11.13. It inherits from the standard TreeMap class and relies on the balancing framework described in Section 11.2.1. We highlight two important aspects of our implementation. First, the AVLTreeMap overrides the definition of the nested _Node class, as shown in Code Fragment 11.12, in order to provide support for storing the height of the subtree stored at a node. We also provide several utilities involving heights of nodes, and the corresponding positions.

To implement the core logic of the AVL balancing strategy, we define a utility, named _rebalance, that suffices as a hook for restoring the height-balance property after an insertion or a deletion. Although the inherited behaviors for insertion and deletion are quite different, the necessary post-processing for an AVL tree can be unified. In both cases, we trace an upward path from the position p at which the change took place, recalculating the height of each position based on the (updated) heights of its children, and using a trinode restructuring operation if an imbalanced position is reached. If we reach an ancestor with height that is unchanged by the overall map operation, or if we perform a trinode restructuring that results in the subtree having the same height it had before the map operation, we stop the process; no further ancestor's height will change. To detect the stopping condition, we record the "old" height of each node and compare it to the newly calculated height.

```
1  class AVLTreeMap(TreeMap):
2    """Sorted map implementation using an AVL tree."""
3
4    #------------------------- nested _Node class -------------------------
5    class _Node(TreeMap._Node):
6      """Node class for AVL maintains height value for balancing."""
7      __slots__ = '_height'          # additional data member to store height
8
9      def __init__(self, element, parent=None, left=None, right=None):
10       super().__init__(element, parent, left, right)
11       self._height = 0               # will be recomputed during balancing
12
13     def left_height(self):
14       return self._left._height if self._left is not None else 0
15
16     def right_height(self):
17       return self._right._height if self._right is not None else 0
```

Code Fragment 11.12: AVLTreeMap class (continued in Code Fragment 11.13).

```
18    #------------------------ positional-based utility methods ------------------------
19    def _recompute_height(self, p):
20      p._node._height = 1 + max(p._node.left_height( ), p._node.right_height( ))
21
22    def _isbalanced(self, p):
23      return abs(p._node.left_height( ) − p._node.right_height( )) <= 1
24
25    def _tall_child(self, p, favorleft=False): # parameter controls tiebreaker
26      if p._node.left_height( ) + (1 if favorleft else 0) > p._node.right_height( ):
27        return self.left(p)
28      else:
29        return self.right(p)
30
31    def _tall_grandchild(self, p):
32      child = self._tall_child(p)
33      # if child is on left, favor left grandchild; else favor right grandchild
34      alignment = (child == self.left(p))
35      return self._tall_child(child, alignment)
36
37    def _rebalance(self, p):
38      while p is not None:
39        old_height = p._node._height          # trivially 0 if new node
40        if not self._isbalanced(p):           # imbalance detected!
41          # perform trinode restructuring, setting p to resulting root,
42          # and recompute new local heights after the restructuring
43          p = self._restructure(self._tall_grandchild(p))
44          self._recompute_height(self.left(p))
45          self._recompute_height(self.right(p))
46        self._recompute_height(p)             # adjust for recent changes
47        if p._node._height == old_height:     # has height changed?
48          p = None                            # no further changes needed
49        else:
50          p = self.parent(p)                  # repeat with parent
51
52    #------------------------- override balancing hooks -------------------------
53    def _rebalance_insert(self, p):
54      self._rebalance(p)
55
56    def _rebalance_delete(self, p):
57      self._rebalance(p)
```

Code Fragment 11.13: AVLTreeMap class (continued from Code Fragment 11.12).